

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 03 -
**ESTRATÉGIAS DE
ATUALIZAÇÃO E
ROLLOUTS**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	16
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	22

EMANIP

O QUE VEM POR AÍ?

Atualizar uma aplicação em produção é sempre um momento delicado. Queremos introduzir novas versões com correções e funcionalidades, mas sem interromper o serviço para os usuários e sem causar inconsistências (como metade dos pods falando linguagem antiga e metade linguagem nova). No ambiente Kubernetes, a entidade responsável por gerenciar atualizações da maioria das workloads é o **Deployment**, que suporta diferentes **estratégias de atualização (Rollout)**.

Nesta aula, vamos aprofundar duas estratégias principais – **Rolling Update** (atualização contínua) e **Recreate** – entender parâmetros como `maxSurge` e `maxUnavailable` que ajustam o comportamento do rollout, e explorar o que acontece durante um **rollout** (desenrolar da atualização). Também veremos os comandos úteis do Kubernetes para monitorar e controlar deploys: `kubectl rollout status`, `history` e `undo`.

Para contextualizar, imagine um cenário: você tem 100 pods rodando a versão 1 de um serviço web e quer lançar a versão 2 sem derrubar o serviço. O *Rolling Update* padrão do Kubernetes consegue exatamente isso – ele vai substituindo os pods gradualmente, alguns de cada vez, de forma que o serviço continue atendendo (talvez com 90% pods v1 e 10% pods v2 em certo momento, mas sempre algo respondendo).

Por outro lado, se tivesse um caso em que a v1 e v2 **não podem rodar simultaneamente** (talvez pelo banco de dados precisar de upgrade incompatível), você pode usar a estratégia *Recreate*: para a versão 1 completamente antes de iniciar a 2 – isso causa um pequeno downtime, mas garante que nunca existirá mistura das versões. Cada abordagem resolve um problema e tem trade-offs: Rolling Updates focam em zero-downtime, Recreate foca em consistência total (ao preço de interrupção).

HANDS ON

Nesta seção hands on, construiremos um simulador em Rust para desmistificar o funcionamento de um Rolling Update do Kubernetes. Nosso código receberá o número de réplicas desejado e os parâmetros `maxSurge` e `maxUnavailable` para orquestrar a atualização.

O programa simulará o processo passo a passo, calculando a cada iteração quantos novos Pods podem ser criados e quantos antigos devem ser removidos, sempre respeitando os limites definidos para garantir a disponibilidade.

Ao executar a simulação, você acompanhará um log que mostra como o número total de Pods pode crescer temporariamente (efeito do `maxSurge`) e como a quantidade de Pods disponíveis nunca cai abaixo do mínimo permitido (graças ao `maxUnavailable`). Este exercício prático revela o algoritmo exato que o Kubernetes usa para realizar atualizações sem indisponibilidade (zero downtime).

SAIBA MAIS

A estratégia **RollingUpdate** é o padrão em Deployments no Kubernetes porque formaliza, no próprio objeto de alto nível, a ideia de “mudar sem parar”, isto é, aproximar gradualmente o estado real do estado desejado sem suspender o serviço.

O controlador de Deployments observa a especificação, detecta alterações no template de Pod e, a cada reconciliação, cria um **ReplicaSet** para a versão atual enquanto reduz proporcionalmente o **ReplicaSet** anterior, de modo que a substituição aconteça a uma taxa controlada. Essa dinâmica tem implicações verificáveis: um rollout só é disparado quando há mudança no campo `.spec.template`; operações como “escalar para cima ou para baixo” não criam revisões nem iniciam um novo ciclo de substituição.

A página conceitual de Deployments explicita essa semântica e ilustra, inclusive, que o algoritmo garante simultaneamente um piso de disponibilidade e um teto de pods simultâneos. Por exemplo, com quatro réplicas e parâmetros padrão, o sistema “garante pelo menos três disponíveis” e “no máximo quatro em execução” durante a transição. Em termos práticos, é essa combinação de reconciliação contínua, disparo por mudança no template e garantias de piso/teto que torna **RollingUpdate** o default de estratégia para Deployments.

O próprio objeto Deployment codifica duas estratégias exclusivas: **RollingUpdate**, como padrão, e **Recreate**, como alternativa em que todos os Pods antigos são terminados antes da criação de novos. A estratégia **RollingUpdate** é parametrizada por dois limites declarativos, **maxUnavailable** e **maxSurge**, que podem ser fornecidos como números absolutos ou percentuais do número desejado de réplicas.

Esses parâmetros têm regras de arredondamento diferentes — indisponibilidade arredonda para baixo, excedente arredonda para cima — e uma restrição estrutural: não é permitido configurar ambos como zero ao mesmo tempo. O valor padrão para ambos é vinte e cinco por cento, o que induz precisamente o comportamento ilustrado na documentação: com N réplicas, durante o rollout haverá, no mínimo, aproximadamente $(1 - 0,25) \cdot N$ pods disponíveis.

e, no pico, até $(1 + 0,25) \cdot N(1 + 0,25) \cdot N$ pods em execução, sempre respeitando os arredondamentos definidos. Essa leitura direta dos parâmetros explica por que o Kubernetes consegue manter, durante a atualização, um trade-off explícito entre continuidade de atendimento e velocidade de troca.

Além desses limites de disponibilidade e capacidade, o ciclo de vida do rollout é observável e controlável por linhas de comando padronizadas. O histórico de revisões é mantido automaticamente pelo controlador por meio dos **ReplicaSets** gerados, e pode ser inspecionado com o subcomando de histórico; a qualquer momento é possível acompanhar o progresso de uma atualização em execução com o subcomando de status, que retorna código de saída diferente de zero quando a condição de progresso foi excedida; e, se necessário, é viável desfazer a atualização revertendo para uma revisão anterior com o subcomando de undo.

Para manter o histórico manejável e, ao mesmo tempo, garantir reversões previsíveis, o Kubernetes expõe o campo **spec.revisionHistoryLimit**, cujo padrão é dez, iniciando a coleta de lixo de ReplicaSets antigos apenas depois que o Deployment atinge o estado de “completo”.

Complementarmente, o campo **progressDeadlineSeconds** define a janela de tolerância para estagnação do rollout e, por padrão, vale 600 segundos; se a atualização não avança nesse intervalo, o recurso é marcado com a razão **ProgressDeadlineExceeded**, e o comando de status reporta essa condição. Esses comportamentos (histórico, status, undo, limite de histórico e janela de progresso) constituem a interface operativa que permite auditar, pausar, retomar, reverter e diagnosticar rollouts de forma determinística.

Para efeito de modelagem, considere um Deployment com alvo de N réplicas e parâmetros $u = \text{maxUnavailable}$ e $s = \text{maxSurge}$, definidos como frações (por exemplo, 0,25 para 25%). Em cada instante, o número de Pods disponíveis não cai abaixo de $(1 - u) \cdot N$; simetricamente, o número total de Pods em execução (versão antiga + versão nova) não excede $(1 + s) \cdot N$.

Essas desigualdades são propriedades invariantes do controlador durante RollingUpdate: sempre que o processo cria Pods novos ele respeita o limite superior imposto por **maxSurge**, e sempre que elimina Pods antigos respeita o limite inferior imposto por **maxUnavailable**.

Como consequência, para $u = 0$ tem-se a cota inferior de disponibilidade igual a N , implicando atualização sem queda de capacidade mínima; para $s = 0$ não há excedente temporário de Pods, e a troca precisa ocorrer inteiramente “dentro do orçamento atual” de N . Os valores-padrão do Kubernetes — 25% para ambos — implicam disponibilidade mínima de 75% e pico máximo de 125% do alvo.

A dinâmica temporal do rollout depende de três fatores acoplados: o tamanho efetivo de cada “lote” de criação/remoção que o controlador consegue executar dado (u, s) e o estado do cluster; o tempo de inicialização de cada Pod novo (tempo até atingir Ready) acrescido do tempo mínimo de prontidão exigido pelo spec; e o tempo de terminação dos Pods antigos.

Para capturar isso com precisão, modelamos o passo elementar do RollingUpdate como um ciclo de duas fases: criação de até $C \uparrow = \lceil sN \rceil$ Pods novos (respeitando o teto de $(1 + s) \cdot N$ em execução) seguida da remoção de até $C \downarrow = \lfloor uN \rfloor$ Pods antigos (respeitando o piso de $(1 - u) \cdot N$ disponíveis).

A execução alterna essas fases sob a regra causal “só remova após validar disponibilidade dos novos”, o que introduz uma latência de gating vinculada a readiness. O Kubernetes formaliza esse gating com **minReadySeconds**, exigindo que cada Pod novo permaneça “Ready” por um intervalo mínimo antes de ser considerado “Available” pelo controlador, o que reduz falsos positivos e evita oscilações na presença de balanceadores de carga com propagação lenta; o limite de progresso **progressDeadlineSeconds** determina quanto tempo o controlador espera por avanço antes de marcar a condição de progresso excedido, sinal útil para automação de rollback ainda que o próprio Deployment não reverta por conta própria.

Com esse esqueleto, é possível derivar limites inferiores e superiores para o “makespan” do rollout, isto é, o tempo total até concluir a substituição de N réplicas. Seja T_i a variável aleatória “tempo até Available” do i -ésimo Pod novo, que inclui pull de imagem, start do processo, readiness e a janela **minReadySeconds**. Admitindo que os T_i são i.i.d. com função de distribuição F e que os Pods são criados por lotes de tamanho $k \leq C \uparrow$, o tempo para completar um lote é o máximo das k amostras, cuja esperança é:

$$E[\max\{T_1, \dots, T_k\}] = \int_0^\infty (1 - F(t)^k) dt$$

Em particular, se T fosse exponencial com taxa λ , teríamos:

$$E[\max_k] = \frac{H_k}{\lambda}$$

Com H_k número harmônico; no mundo real, T costuma ter cauda mais pesada que exponencial por conta de downloads de imagem e variações de I/O, razão pela qual o uso de quantis (p95/p99) é preferível para limites operacionais. Uma aproximação útil para o número de iterações é passos $\approx \lceil \frac{N}{\min\{C \uparrow, C \downarrow\}} \rceil$, assumindo que cada ciclo líquido converte pelo menos $\min\{C \uparrow, C \downarrow\}$ réplicas antigas em novas. O efeito de **minReadySeconds** adiciona uma constante m por passo ao tempo esperado, que pode ser incorporada como deslocamento à distribuição T ; o tempo de terminação contribui com uma constante adicional, mas o gargalo típico é readiness dos novos.

A disponibilidade percebida durante o rollout pode ser quantificada como fração de capacidade ativa. Supondo que cada réplica contribua igualmente para throughput e que o balanceador distribua tráfego de forma proporcional ao número de réplicas Ready, a capacidade instantânea mínima é $(1 - u) \cdot N$ réplicas, logo a disponibilidade relativa mínima é $(1 - u)$.

No plano de SRE, a disponibilidade média em uma janela C é $A = \left(\frac{\text{uptime}}{\text{uptime} + \text{downtime}} \right)$, e relaciona-se a MTBF/MTTR por $A \approx \frac{MTBF}{(MTBF + MTTR)}$ sob hipóteses de renovação; a tabela canônica de “nines” fornece a tradução para orçamentos de erro em minutos por período. Esses conceitos ajudam a escolher (u, s) : se o orçamento semanal de indisponibilidade é, por exemplo, 10,08 minutos para SLO de 99,9%, pode-se restringir u para que a degradação de *throughput* prevista, multiplicada pela duração estimada do rollout por passo, não consuma o orçamento.

Uma análise de filas deixa esse raciocínio mais explícito. Se a taxa de chegada de requisições é λ e a capacidade de serviço por réplica é μ , com c réplicas efetivamente Ready em determinado instante, a utilização média é $\rho = \frac{\lambda}{c\mu}$ em um modelo M/M/c.

A estabilidade exige $\rho < 1$; durante o rollout, quando c cai para $(1 - u)N$, a utilização sobe para $\rho' = \frac{\lambda}{(1-u)N\mu}$. O backlog cresce sempre que $\rho' \geq 1$. Pela Lei de Little, $L = \lambda W$, de modo que um aumento transitório de W durante a atualização é proporcional ao backlog L acumulado quando a capacidade é comprimida. Daí a motivação para $u = 0$ na janela de pico: manter $c = N$ evita a região supercrítica onde latências explodem. Se, por outro lado, o cluster tem folga e a janela é de baixo tráfego, aumentar u reduz o makespan e pode manter $\rho' < 1$ sem afetar SLO.

Cabe ainda integrar o parâmetro s na análise de recursos. Um s alto acelera o rollout ao aumentar $C \uparrow$, mas eleva temporariamente o número total de Pods em execução para $(1 + s) \cdot N$, pressionando CPU/memória e, indiretamente, serviços compartilhados como bancos e filas. Em arquiteturas com conexões limitadas por licença ou por pool, dobrar Pods pode dobrar conexões e levar o backend à saturação.

Esse teto é a parede contra a qual o escalonador esbarra: se não houver recursos disponíveis, o lote não nasce, alongando o tempo por passo e anulando os ganhos teóricos de um s elevado. Assim, é prudente vincular s a um orçamento de capacidade ociosa medido e validar a resposta do back-end sob carga com testes de pré-produção.

Do ponto de vista algorítmico, escolher (u, s) e o tamanho de cada passo para minimizar o tempo total sob restrições de disponibilidade e de recursos é um problema de scheduling com restrições de capacidade e precedência (criar $\rightarrow$ validar $\rightarrow$ remover). Problemas desse tipo são, em geral, NP-difíceis, o que explica por que o Kubernetes implementa políticas simples, determinísticas e locais em vez de um otimizador global.

Por analogia, mesmo o subproblema clássico de “encaixar” trabalho em bins de capacidade finita (*bin packing*) é fortemente NP-completo e admite apenas esquemas aproximativos com limites conhecidos. O rollout herda essa natureza combinatória quando tentamos orquestrar lotes sob múltiplas restrições; portanto, recorre-se a heurísticas de alto nível com garantias simples e auditáveis.

Três estratégias práticas ilustram escolhas de projeto. A “Disponibilidade-Primeiro” fixa $u = 0$ e escolhe s máximo tal que $(1 + s) \cdot N$ caiba no orçamento de recursos e não sobrecarregue serviços dependentes; a cada etapa,

cria-se $C \uparrow$ réplicas novas, aguarda-se readiness por um tempo de guarda definido por quantis do máximo de readiness times, valida-se SLIs e, somente então, remove-se exatamente $C \uparrow$ réplicas antigas.

A seguir vai um exemplo “Disponibilidade-Primeiro” nativo com Deployment (Kubernetes puro), onde **maxUnavailable**: 0 garante que réplicas antigas só serão removidas depois que as novas estiverem “Available”; **minReadySeconds** atua como tempo de guarda; e **readinessGates** permite que um controlador externo marque uma condição customizada (por exemplo, “SLIHealthy”) antes de o Pod ser considerado disponível:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  labels:
    app: myapp
  annotations:
    kubernetes.io/change-cause: "Rollout v2.0.0 – estratégia
disponibilidade-primeiro"
spec:
  replicas: 10
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 600 # janela de progresso
padrão (10min)
  minReadySeconds: 30
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0 # u = 0 → nunca reduza a
capacidade mínima
      maxSurge: "50%"
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      readinessGates:
        - conditionType: example.com/SLIHealthy
      containers:
        - name: app
          image: ghcr.io/suaorg/myapp:2.0.0
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 3
```

A estratégia “*Tempo-Mínimo sob Orçamento de Recurso*” parte de um orçamento B para capacidade total temporária, isto é, exige $(1 + s)N \leq B$, e de um

limite de degradação admissível D , isto é, $(1 - u)N \geq D$; escolhem-se s e u nos limites desses vínculos para maximizar $\min\{C \uparrow, C \downarrow\}$ e minimizar o número de passos.

No exemplo, definimos replicas: 12 ($N = 12$). Suponha um orçamento de capacidade de pico $B = 18$ Pods e um limite de capacidade mínima $D = 9$ Pods disponíveis durante a troca. Então:

$$s = \min\left\{\frac{B}{N} - 1, 1\right\} = \frac{18}{12} - 1 = 0,5 \Rightarrow \text{maxSurge: 50\%}$$

$$u = 1 - \frac{D}{N} = 1 - \frac{9}{12} = 0,25 \Rightarrow u = 1 - \frac{D}{N} = 1 - \frac{9}{12} = 0,25 \Rightarrow \text{maxUnavailable: 25\%}$$

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  labels: { app: myapp }
  annotations:
    kubernetes.io/change-cause: "Rollout v2.0.0 - tempo-mínimo sob
orçamento"
spec:
  replicas: 12      # N
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 600
  minReadySeconds: 15  # tempo de guarda; ajuste ao seu p95 de readiness
  strategy:
    type: RollingUpdate
    rollingUpdate:
      # Saturar os vínculos: (1+s)N <= B e (1-u)N >= D
      maxSurge: "50%"      # s* = B/N - 1  → aqui, B=18, N=12  → 50%
      maxUnavailable: "25%" # u* = 1 - D/N  → aqui, D=9, N=12  → 25%
  selector:
    matchLabels: { app: myapp }
  template:
    metadata:
      labels: { app: myapp }
    spec:
      containers:
        - name: app
          image: ghcr.io/suaorg/myapp:2.0.0
          ports: [ { containerPort: 8080 } ]
          readinessProbe:
            httpGet: { path: /healthz, port: 8080 }
            periodSeconds: 5
            failureThreshold: 3
```

A estratégia “*Quantil-Gated Canary por Lotes*” começa com um lote canário $k_0 \ll N$, com u e s baixos, define portas de promoção por quantis de métricas e de readiness e, a cada aprovação, aumenta geometricamente o tamanho do lote até saturar $C \uparrow$, elevando gradualmente u conforme a confiança cresce; diante de falha, pausa-se e executa-se undo. O Rollout começa com um lote canário pequeno (setWeight: 5), roda uma análise inline baseada em quantis (ex.: p95 de latência via histogram_quantile + taxa de erro), e só então promove para o próximo lote. Os lotes crescem geometricamente ($5 \rightarrow 10 \rightarrow 20 \rightarrow 40 \rightarrow 80 \rightarrow 100$).

Em falha de análise, o rollout é **abortado/rollback** automaticamente. maxUnavailable: 0% mantém disponibilidade (remoção só ocorre após o lote passar no gate); maxSurge é baixo no começo e você pode elevá-lo em rollouts futuros conforme a confiança crescer. Com trafficRouting.nginx o setWeight faz split real de tráfego; se você não tiver *traffic router*, o Argo usa **réplicas ponderadas** para aproximar a porcentagem (sem mesh/ingress) — ambos comportamentos são de fábrica. Por brevidade, apresentam-se somente as políticas de *spec* para os *rollouts*:

```
spec:
  replicas: 20
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 900
  strategy:
    canary:
      maxUnavailable: "0%"
      maxSurge: "10%"
      abortScaleDownDelaySeconds: 300
      canaryService: myapp-canary
      stableService: myapp-stable
      trafficRouting:
        nginx:
          stableIngress: myapp-ing
      steps:
        - setCanaryScale:
            matchTrafficWeight: true
        - setWeight: 5
        - pause: { duration: 60s }
        - analysis:
            templates:
              - templateName: slo-quantile-check
            args:
              - name: error_rate_threshold
                value: "0.01" # 1% máx
              - name: p95_latency_threshold_seconds
                value: "0.300" # p95 < 300 ms
        - setWeight: 10
        - pause: { duration: 60s }
        - analysis:
            templates: [ { templateName: slo-quantile-check } ]
        - setWeight: 20
        - pause: { duration: 60s }
        - analysis:
            templates: [ { templateName: slo-quantile-check } ]
        - setWeight: 40
        - pause: { duration: 60s }
        - analysis:
            templates: [ { templateName: slo-quantile-check } ]
        - setWeight: 80
        - pause: { duration: 60s }
        - analysis:
            templates: [ { templateName: slo-quantile-check } ]
        - setWeight: 100
        - pause: { duration: 30s }
  selector:
    matchLabels: { app: myapp }
```

A fronteira entre **RollingUpdate** e **Recreate** pode ser descrita em termos de compatibilidade e de custo temporal. Se versões não podem coexistir, a disponibilidade durante a transição é zero e o makespan é a soma do tempo de desligamento da versão antiga com o readiness da nova.

Embora simples de raciocinar, esse plano consome todo o orçamento de erro durante a janela, o que raramente é aceitável em sistemas interativos; por isso, se **Recreate** for inevitável, recorrem-se a janelas de baixo tráfego e a testes pródigos de pré-produção. A alternativa blue/green desloca a decisão crítica para o plano de roteamento: mantêm-se dois conjuntos de Pods isolados, promove-se tráfego quando o green estiver saudável e reverte-se quase instantaneamente trocando o Service caso necessário.

Em Kubernetes, padrões como blue/green e canário não são nativos do Deployment, mas podem ser declarados e automatizados por controladores como Argo Rollouts, que adicionam gates de promoção e deadlines de progresso sobre conceitos equivalentes.

Por fim, um método de projeto em três etapas ajuda a transformar melhores práticas em política quantificada: derive limites de disponibilidade e capacidade com (u, s) e valide-os contra orçamentos de erro do SLO; estime o makespan por lotes usando estatísticas de ordem dos tempos de readiness e escolha k respeitando $C \uparrow$; selecione o algoritmo conforme o momento do negócio, lembrando que otimizadores globais exatos são impraticáveis.

Em cenários de pico, $u = 0$ e s generoso, mas dentro do orçamento de recursos, tendem a ser a escolha dominante; em janelas de baixa demanda, u maior reduz passos e tempo total; quando a compatibilidade não existe, Recreate ou blue/green deslocam o trade-off para um downtime curto e planejado ou para duplicação de recursos com comutação atômica.

Em todos os casos, a clareza matemática sobre disponibilidade, tempo de readiness e tamanho de lote transforma uma prática em uma política justificável, auditável e adequada ao risco do negócio.

MERCADO, CASES E TENDÊNCIAS

Imagina que o seu cluster é uma cidade viva e barulhenta. A cada esquina há um serviço servindo gente, e seu trabalho é trocar as lâmpadas da iluminação pública sem jamais apagar a rua. Na avenida principal mora o Deployment, aquele síndico que sabe trocar lâmpadas com elegância: ele cria um novo conjunto de réplicas enquanto reduz o antigo, e faz isso no ritmo que você configura com dois botões — `maxSurge` e `maxUnavailable`. É a coreografia clássica do `RollingUpdate`, documentada com todas as letras: você descreve o estado desejado, o controlador aproxima o mundo real do ideal a uma taxa controlada, e ainda te dá comandos para ver status, reverter e consultar histórico. É o beabá, mas continua sendo a raiz das histórias de rollout no mundo real.

Nos últimos anos, porém, a cidade pediu mais: além de trocar lâmpadas sem escurecer a rua, queríamos testar a lâmpada nova em um quarteirão, medir a satisfação da vizinhança, e só então expandir. Daí nasceu a pauta de *progressive delivery* — expor a nova versão a uma fração pequena de usuários, observar métricas e ampliar gradualmente. Foi esse o espírito que levou o projeto Argo a lançar o `Argo Rollouts` como controlador de canary e blue/green com gates automáticos, consolidando uma prática que já se desenhava em muitos times. A narrativa oficial crava: rollouts graduais mitigam risco porque a promoção é condicionada a sinais de saúde claros, reduzindo “big bang” e melhorando MTTR quando algo sai do trilho.

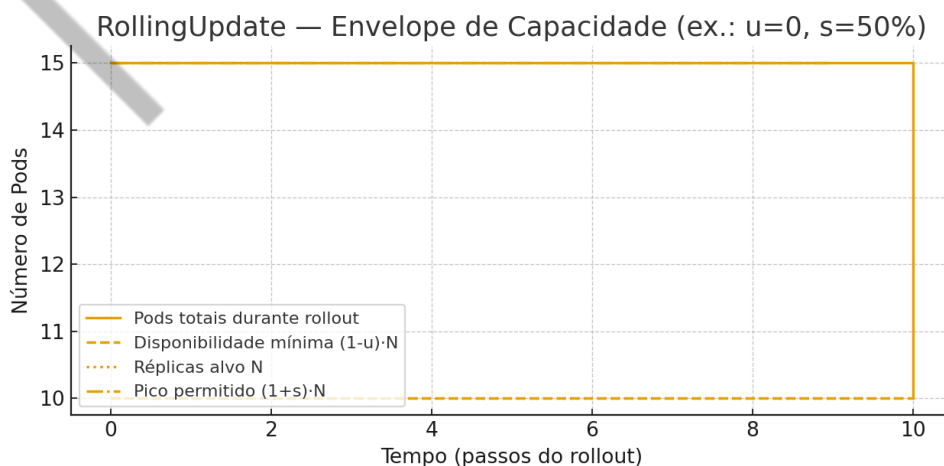


Figura 1 – RollingUpdate
Fonte: Elaborado pelo autor (2025)

Se você quer um pôster para a parede, a turma da Netflix publicou o Kayenta, análise canário automatizada que compara estatisticamente “baseline vs. canário” e decide se há degradação significativa. O texto do TechBlog detalha a lógica: reduzir o julgamento humano a um “gate” com evidência de métricas, e o anúncio conjunto com o Google destacou a abertura do projeto e a ambição de fazer deploy em alta velocidade sem perder segurança. Marcas como a Adobe contaram como integraram o Kayenta para fortalecer a própria esteira e até deram um passo além, acrescentando detecção de anomalias com aprendizado de máquina para logs de erro — uma amostra de como canary analysis virou peça central em organizações de larga escala.

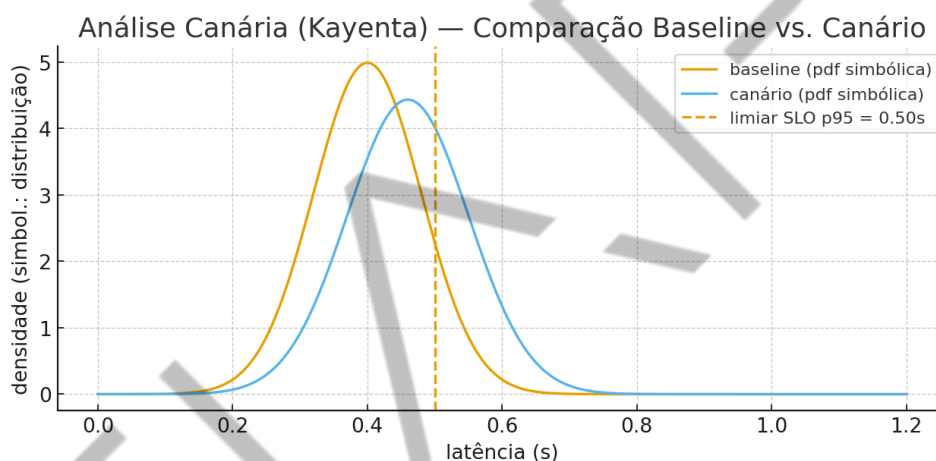


Figura 2- Análise Canária
Fonte: Elaborado pelo autor (2025)

Do outro lado da cidade, o comércio é quem mais sofre com janelas críticas. Varejistas descrevem o quebra-cabeça de operar centenas ou milhares de pontos numa borda distribuída, onde cada loja roda seu próprio mini-cluster e as atualizações precisam ser coordenadas, previsíveis e reversíveis. A CNCF mapeou desafios típicos dessa adoção em retail-edge — de padronização a observabilidade —, e a história da Chick-fil-A virou referência: milhares de restaurantes executando Kubernetes na borda, com serviços de plataforma para implantação e observabilidade por cima. No dia a dia isso se traduz em lotes, canários por grupos de lojas e comutação segura, porque a pior hora para errar é justamente quando a fila está maior.

Rollout em Varejo na Borda — Coortes de Lojas

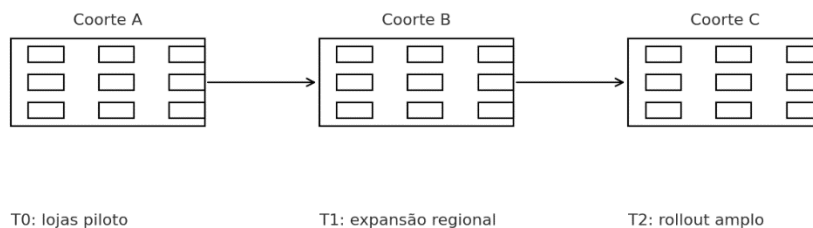


Figura 3 – Rollout em Varejo na Borda
Fonte: Elaborado pelo autor (2025)

Em termos de “moda que pegou”, dá para enxergar três movimentos. O primeiro é o de times que saem do RollingUpdate puro e abraçam ferramentas especializadas para gates automáticos. O blog do projeto Argo crava a definição e a motivação do progressive delivery, e vários relatos de engenharia mostram a adoção prática de Rollouts como a peça que faltava ao trio Kubernetes + GitOps + observabilidade. O segundo é a normalização da comparação entre Flagger e Argo Rollouts, principalmente em ambientes com service mesh como Linkerd: a discussão já não é “se” usar progressive delivery, mas “qual” é o melhor encaixe para o stack. O terceiro é a chegada de guias cada vez mais pragmáticos que ligam a configuração de maxSurge/maxUnavailable ao efeito real na disponibilidade e no tempo de rollout — algo que antes ficava como conhecimento tácito nos times SRE.

Laço de Controle — Flagger + Linkerd (visão conceitual)

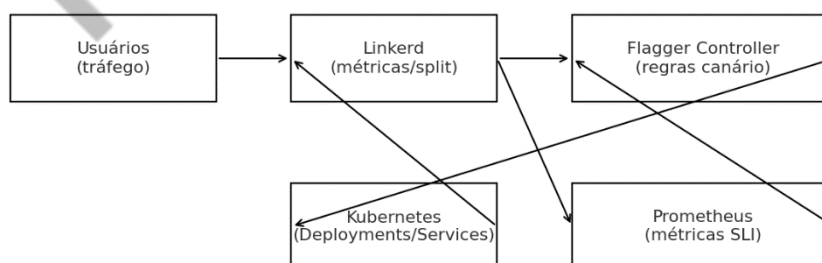


Figura 4 - Laço de Controle
Fonte: Elaborado pelo autor (2025)

Na prática, o roteiro de um canário maduro parece um conto com capítulos e checkpoints. Primeiro, você define lotes pequenos, mede p95 de latência e taxa de erro, e só então torna a história mais ambiciosa. Depois, usa “pausas” como cliffhangers: um tempo de guarda para o ambiente assentar e para os dashboards te contarem se a audiência gostou do episódio. Por fim, promove gradualmente até “100%” — e tudo com a rede de segurança do rollback, que hoje costuma ser um clique na ferramenta (ou um undo no Kubernetes). Na literatura de progressive delivery, esse encadeamento virou padrão: steps que ajustam peso, análises automáticas que avaliam métricas e políticas que abortam a promoção quando os números fogem da margem. Quem lê os tutoriais e estudos de caso nota que esse padrão está se consolidando como “linguagem franca” para releases.

A borda traz seu tempero especial. Quando trocamos uma versão em milhares de lojas, a brincadeira deixa de ser “cinco pods no cluster X” e passa a ser “cinco mil endpoints num mapa”. O ecossistema descreve esse cenário como um caso extremo de coordenação e GitOps: pipelines disparam lotes por coortes, telemetria central consolida o canário geográfico, e a reversão devolve o tráfego para o conjunto estável em segundos. O detalhe saboroso é que, na borda, a resiliência local manda: as lojas seguem operando mesmo sem internet, então a atualização precisa ser conservadora e suplantando as surpresas de rede intermitente. Nesses relatos, a técnica é a estrela, mas a direção de fotografia é da plataforma: padronização dos manifests, canais de release, e “pontos de fuga” bem definidos.

E onde entra o velho RollingUpdate da documentação? Ele continua sendo seu instrumento de corda: afinação confiável, sem firula. Entender o efeito de maxSurge e maxUnavailable evita dois tropeços clássicos. Se maxUnavailable for alto demais, você acelera os passos, mas pode pressionar a fila de requisições (menos pods atendendo a mesma carga). Se maxSurge for alto demais, você acelera a criação, mas pode sufocar o banco e as filas com conexões duplicadas. Guias técnicos mostram exatamente essa balança: maxSurge é potência, maxUnavailable é freio — mexer em ambos exige ouvir o “motor” do back-end.

Fechando a caminhada, vale lembrar que a “ficção” aqui tem rigor estatístico. As histórias de Netflix e Adobe com Kayenta ensinaram o mercado a trocar “feeling” por hipótese + teste: comparar canário e baseline por métricas, definir thresholds e automatizar a decisão de seguir ou voltar. O Argo Rollouts transformou essa prática

em CRDs que você versiona no Git; o resto da pilha — Prometheus, Grafana, OpenTelemetry — virou o conjunto de sensores do seu palco. E a moral da fábula é simples: quanto mais frequentes e pequenas as mudanças, mais importante é ter um roteiro de rollout que trate erro como personagem recorrente, não como vilão surpresa. Assim, quando o plot twist vier, você pausa, mede, volta uma cena, reescreve a linha e segue a temporada.



O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a gerenciar atualizações no Kubernetes. Vimos como a estratégia padrão RollingUpdate de um Deployment controla a transição entre versões, usando os parâmetros `maxUnavailable` e `maxSurge` para garantir disponibilidade e limitar o uso de recursos. Exploramos também a estratégia Recreate como uma alternativa com downtime.

Na prática, você aprendeu a usar os comandos `kubectl rollout status`, `history` e `undo` para monitorar, auditar e reverter atualizações. Por fim, conectamos esses fundamentos a estratégias avançadas como Canary e Blue/Green, capacitando você a projetar rollouts rápidos, seguros e controlados. **Bons deploys!**

REFERÊNCIAS

Argo Project. **'Analysis & Progressive Delivery'**. Argo Rollouts Documentation. 2025. Disponível em: <https://argo-rollouts.readthedocs.io/en/stable/features/analysis/> Acesso em 04 set. 2025.

Argo Project. **'Getting Started — NGINX Ingress'**. Argo Rollouts Documentation. 2025. Disponível em: <https://argo-rollouts.readthedocs.io/en/stable/getting-started/nginx/> Acesso em 04 set. 2025.

Argo Project. **'NGINX Traffic Management'**. Argo Rollouts Documentation. 2025. Disponível em: <https://argo-rollouts.readthedocs.io/en/stable/features/traffic-management/nginx/> Acesso em 04 set. 2025.

Argo Project. **'Prometheus Metrics for Analysis'**. Argo Rollouts Documentation. 2025. Disponível em: <https://argo-rollouts.readthedocs.io/en/stable/analysis/prometheus/> Acesso em 04 set. 2025.

Chambers, B. and Crane, A. **'Observability at the Edge: How Chick-fil-A provides observability for 2,800+ K8s clusters'**. Chick-fil-A Tech Blog (Medium). 2023. Disponível em: <https://medium.com/chick-fil-atech/observability-at-the-edge-b2385065ab6e> Acesso em: 04 set. 2025.

Chambers, B., Hurd, C. and Crane, A **'Bare Metal K8s Clustering at Chick-fil-A Scale'**. Chick-fil-A Tech Blog (Medium). 2025. Disponível em: <https://medium.com/chick-fil-atech/bare-metal-k8s-clustering-at-chick-fil-a-scale-929a0e6d29e5> Acesso em: 04 set. 2025.

Chick-fil-A Team. **'Hundreds of Weekly Deployments Managed with GitOps'**. Chick-fil-A Tech Blog (Medium). 2022. Disponível em: <https://medium.com/chick-fil-atech/hundreds-of-weekly-deployments-managed-with-gitops-51f6cc60c6b3> Acesso em: 04 set. 2025.

CNCF. **'4 Challenges Retailers Face When Adopting Kubernetes at the Edge'**. Cloud Native Computing Foundation Blog. 2022. Disponível em: <https://www.cncf.io/blog/2022/09/08/4-challenges-retailers-face-when-adopting-kubernetes-at-the-edge/> Acesso em: 04 set. 2025.

Ehmke, M. **'How We Automated Canary Analysis for Deployments'**. Adobe Tech Blog (Medium). 2019. Disponível em: <https://medium.com/adobetech/how-we-automated-canary-analysis-for-deployments-6a7ff88e4b7e> Acesso em: 04 set. 2025.

Flagger Maintainers (FluxCD). **'Linkerd Canary Deployments with Flagger'**. Flagger Documentation. 2023. Disponível em: <https://docs.flagger.app/tutorials/linkerd-progressive-delivery> Acesso em: 04 set. 2025.

Flagger Maintainers (FluxCD). 'Flagger — **Introduction**'. Flagger Documentation. 2024. Disponível em: <https://docs.flagger.app/> Acesso em: 04 set. 2025.

Graff, M. and Sanden, C. '**Automated Canary Analysis at Netflix with Kayenta**'. Netflix Technology Blog. 2018. <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69> Acesso em 04 set. 2025.

Linkerd Authors. '**Progressive Delivery (Flagger and Argo Rollouts)**'. Linkerd Documentation. 2025. Disponível em: <https://linkerd.io/2-edge/tasks/flagger/> Acesso em 04 set. 2025.

Phillips, A. '**Introducing Kayenta: An Open Automated Canary Analysis Tool from Google and Netflix**'. Google Cloud Blog. 2018. Disponível em: <https://cloud.google.com/blog/products/gcp/introducing-kayenta-an-open-automated-canary-analysis-tool-from-google-and-netflix> Acesso em: 04 set. 2025.

The Kubernetes Authors. '**kubectl rollout**'. Kubernetes Documentation (kubectl reference). 2024. Disponível em: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_rollout/ Acesso em: 04 set. 2025.

The Kubernetes Authors. '**Deployments**'. Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/> Acesso em: 04 set. 2025.

The Kubernetes Authors. '**EndpointSlices**'. Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/> Acesso em: 04 set. 2025.

The Kubernetes Authors. '**kubectl rollout history**'. Kubernetes Documentation (kubectl reference). 2025. Disponível em: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_rollout/kubectl_rollout_history/ Acesso em: 04/ set. 2025.

The Kubernetes Authors. '**kubectl rollout undo**'. Kubernetes Documentation (kubectl reference). 2025. Disponível em: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_rollout/kubectl_rollout_undo/ Acesso em: 04 set. 2025.

The Kubernetes Authors. '**kube-proxy**'. Kubernetes Documentation (CLI reference). 2025. Disponível em: <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/> Acesso em: 04 set. 2025.

The Kubernetes Authors. '**Service**'. Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/services-networking/service/> Acesso em: 04 set. 2025.

The Kubernetes Authors. **'Virtual IPs and Service Proxies'**. Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/reference/networking/virtual-ips/> Acesso em: 04 set. 2025.

EMANIP

PALAVRAS-CHAVE

Kubernetes. Rollout. Canary.

EMSE



POS TECH